Part A

A1：タブ切り替えUI

HTMLとCSSのみを使用して、タブ切り替えUIを作成しなさい。

- タブは「Tab1」「Tab2」「Tab3」の3つを横並びで表示すること
- 各タブをクリックすると、対応するコンテンツ領域のみが表示され、それ以外は非表示になること
- 初期状態では「Tab1」のコンテンツが表示されていること
- アクティブなタブは他のタブと見た目が異なること（背景色・下線など任意）
- JavaScriptは一切使用しないこと（`**:checked**`擬似クラスや`**:target**`擬似クラスを活用すること）

A2：レスポンシブカードグリッド

CSS Gridを使用して、レスポンシブなカードグリッドを作成しなさい。

- カードを6枚表示すること（内容は任意）
- スマホ(430px以下)・タブレット(1024px以下)・PCの画面幅に応じてカラム数がそれぞれ1列・2列・4列に変化すること
- 各カードには以下の要素を含めること
  - サムネイル画像（`**background-color**` で代替可）
  - タイトル（`**<h3>**` タグ）
  - テキスト（`**<p>**` タグ）
  - ボタン（`**<button>**` タグ）
- カードホバー時にカード全体が強調されること

A3：CSSアニメーション（ローディングスピナー）

CSSのみを使用して、ローディングスピナーを作成しなさい。JavaScriptは使用不可。

- 円形のスピナーが360度回転するアニメーションを実装すること
- アニメーションは無限に繰り返されること
- 回転の速さは1秒で1周とすること
- スピナーの下にテキスト「Loading...」を表示すること
- ページ中央に配置すること

Part B

B1：カウントダウンタイマー

指定した秒数からカウントダウンするタイマー機能を実装しなさい。

- 数値入力フィールド（**`<input type="number">`**）に秒数を入力できること
- 「Start」ボタンをクリックすると、入力した秒数からカウントダウンが始まること
- 表示フォーマットは **`MM:SS`**（例：**`01:30`**）で表示されること
- 「Stop」ボタンでカウントダウンを一時停止できること
- 「Reset」ボタンで初期状態に戻ること
- カウントダウンが **`0:00`** になったら自動停止し、**`alert()`** または画面上に「Time's up!」と表示すること
- カウントダウン中は「Start」ボタンを無効化すること

B2：ドラッグ＆ドロップ並び替えリスト

ドラッグ＆ドロップでリスト項目を並び替えられる機能を実装しなさい。

- リスト項目を5つ表示すること（内容は任意）
- HTML5のドラッグ＆ドロップAPI（**`draggable="true"`**、**`dragstart`**・**`dragover`**・**`drop`** イベント）を使用すること
- ドラッグ中の要素は視覚的に区別できること（**`opacity`** の変化など）
- ドロップ後、リストの順番が変更されて表示されること

B3：テキスト入力バリデーション

フォームの入力値をリアルタイムでバリデーションする機能を実装しなさい。

- 以下の入力フィールドを作成すること
  - ユーザー名（3文字以上20文字以下）
  - メールアドレス（正規表現による形式チェック）
  - パスワード（8文字以上、英字・数字をそれぞれ1文字以上含む）
  - パスワード確認（パスワードと一致しているか）
- 各フィールドから **`blur`** イベント（フォーカスが外れた時）でバリデーションを実行すること

- エラーメッセージはフィールドの直下に赤字で表示すること
- 全フィールドが正常な場合のみ「Submit」ボタンが有効化されること

B4：画像スライダー

JavaScriptを使用して、自動再生機能付きの画像スライダーを実装しなさい。

- スライドを4枚用意すること（**`background-color`** で色分けした **`div`** 要素で代替可）
- 3秒ごとに自動で次のスライドへ切り替わること（**`setInterval`** を使用）
- 「前へ」「次へ」ボタンで手動切り替えもできること
- 手動で切り替えた場合、自動再生のタイマーがリセットされること（**`clearInterval`** + 再 **`setInterval`**）
- 現在のスライド番号を示すインジケーター（ドット）を表示すること
- スライドが最後のものから「次へ」を押すと最初に戻ること（無限ループ）

B5：JSONデータ検索フィルター

JavaScriptのみを使用して、JSONデータをリアルタイムで検索・フィルタリングする機能を実装しなさい。

- 以下の形式のJSONデータを JavaScript 内に直接定義すること（外部ファイル不要）

```
[
  { "id": 1, "name": "田中 太郎", "department": "営業部", "skill": "JavaScript" },
  { "id": 2, "name": "佐藤 花子", "department": "開発部", "skill": "PHP" },
  { "id": 3, "name": "鈴木 一郎", "department": "営業部", "skill": "Python" },
  { "id": 4, "name": "高橋 美咲", "department": "開発部", "skill": "JavaScript" },
  { "id": 5, "name": "伊藤 健太", "department": "総務部", "skill": "CSS" },
  { "id": 6, "name": "渡辺 さくら", "department": "開発部", "skill": "PHP" }
]
```

- テキスト入力フィールドで「名前」または「スキル」をリアルタイム検索できること（**`input`** イベント）
- ドロップダウンで「部署」による絞り込みができること（「全て」を含む）
- テキスト検索と部署フィルターは同時に適用されること（AND条件）
- 検索結果はHTMLテーブルとして表示され、一致する行数も表示されること

- 検索結果が0件の場合「該当するデータがありません」と表示すること

Part C：PHP

C1：フォームバリデーションとXSS対策

PHPを使用して、入力フォームのサーバーサイドバリデーションを実装しなさい。JavaScriptは使用不可。

- 以下のフィールドを持つHTMLフォームを作成すること（**`method="POST"`** で同一ファイルに送信）
  - 氏名（必須、50文字以内）
  - 年齢（必須、1〜120の整数）
  - メールアドレス（必須、形式チェック）
- 送信後、PHPでバリデーションを実行すること
- バリデーションエラーがある場合は、エラーメッセージをフォームの上部に赤字で表示し、入力値を保持したままフォームを再表示すること
- バリデーション通過後、入力された内容を画面に表示すること
- XSS対策として **`htmlspecialchars()`** を使用すること

C2：ファイルへの読み書き（訪問カウンター）

PHPを使用して、ページにアクセスするたびにカウントが増加する訪問カウンターを実装しなさい。JavaScriptは使用不可。

- 訪問回数を **`count.txt`** テキストファイルに保存・読み込みすること
- ページにアクセスするたびにカウンターが +1 されること
- 現在の訪問回数をページ上に表示すること（例：「現在の訪問回数：42回」）
- **`count.txt`** が存在しない場合は自動的に作成し、0からカウントを始めること
- ファイルの読み書きには **`file_get_contents()`** / **`file_put_contents()`** を使用すること
- カウンターを0にリセットするボタンを設置し、フォームのPOST送信でリセット処理を行うこと

Powered by **https://markdowntoimage.com**